Unit 1 - 2 Marks

1. Consider a C program that contains several user-defined functions but does not contain the main() function. Analyze what happens when the program is compiled and executed.
2. A C program contains statements written using C syntax rather than machine instructions. Explain why the program must be translated into machine code before it can be executed by the computer.
3. A programmer attempts to declare a variable as int. Analyze why the compiler does not accept int as a variable name.
4. A student declares a variable using the identifier **2student.** Identify the problem and modify the identifier to make it valid according to C naming rules.
5. A program contains the words int, total, and while. Classify each as a keyword or identifier and justify your classification.
6. Consider the statement scanf("%d", &num);. Explain what information is passed to scanf() through &num and Analyze what could happen if & is omitted.
7. A programmer wants to modify the size and sign properties of an integer data type. Explain how size and sign qualifiers affect the range and storage requirements of the data type, with suitable examples.
8. A program must display "Pass" when a student's marks are 40 or above and "Fail" otherwise. Construct an appropriate if-else structure and explain its execution.
9. A menu-driven program allows the user to select options 1–4 for different operations. Construct a suitable switch control structure and explain how the selected case is executed.
10. A switch statement contains multiple case blocks but no break statements. Analyze the resulting execution and explain why break may be required.
11. A program needs to display the numbers from 1 to 100. Select a suitable looping construct and justify your choice based on the known number of iterations.
12. Consider the following two loops:

```
while(condition)
{
   statement;
}
do
{
   statement;
} while(condition);
```

Analyze their behavior when the condition is initially false and explain the difference in execution.

14. A program needs to select one action based on a student's grade (A, B, C, or D). Compare the suitability of if-else and switch for this problem and justify your choice.
15. A programmer uses an array to store a collection of student records containing roll number, name, and marks. Identify the limitations of using a single array for this purpose and suggest a more appropriate data structure.
16. A program stores the address of variable num in a pointer p. Explain how using p enables the program to access or modify the value of num.
17. Consider the following function:

```
void change(int x)
{
   x = 100;
}
```

If change(num) is called, the value of num remains unchanged. Analyze why this happens in call-by-value and explain how the original value could be modified.

18. A student record contains a roll number, name, marks, and grade. Evaluate why a structure is more appropriate than an array for representing this record.
19. A problem can be solved either by using a loop or by calling a function repeatedly. Compare iteration and recursion in terms of execution, termination condition, and memory usage.
20. Consider the following structure:

```
struct Student
{
   int roll-no;
   float marks;
};
```

and the variable:

```
struct Student s1;
```

Demonstrate how the dot (.) operator can be used to access and modify the roll-no and marks members of s1.

Unit 1 -Essay Questions

1. Discuss how while differs from do- while control structure.
2. Explain how an array differs from a variable. Mention the advantages and disadvantages of arrays.
3. Explain how strings differ from C character data type.

4. Explain how the characteristic features of C contribute to its efficiency, portability, and flexibility. Illustrate any two features with suitable examples.
5. Write a C program to swap two integers using call by value and call by reference .
6. Analyze the sections of a C program and explain the role of each section with a suitable program structure.
7. Apply the rules of C identifiers to identify valid and invalid variable names from a given set and justify your classification.
8. Develop a C program to determine the maximum of four numbers without using an array. Analyze the conditional logic used in the program.
9. Develop three C programs using for, while, and do-while loops to display "GITAM" five times. Compare the execution behavior of the three loops.
10. Develop a C program to generate the Fibonacci series up to a given limit n.
11. Develop a C program to calculate the sum of all even elements in an array of n elements. Analyze how the program identifies even elements.
12. Implement string operations equivalent to strlen(), strcpy(), strcat(), and strcmp() without using the corresponding library functions. Analyze the logic used in each operation.
13. Develop a C program using pointers to store the addresses of an integer and a character variable and display their addresses and values. Analyze the role of pointer data types in accessing these variables
14. Given a function with arguments, identify the actual and formal parameters and explain the flow of values between them during a function call.
15. Analyze a C program that modifies variables through function arguments. Demonstrate the difference between call by value and call by reference using a function to swap two integer values..
16. Given a C program containing local, global, static, and register variables, analyze their scope, lifetime, and accessibility during program execution.
17. Develop a C program to find the roots of a quadratic equation. Analyze the different cases based on the value of the discriminant.
18. Develop a C program to sort a one-dimensional array in ascending order for five elements 3,65,-5,7,34. Analyze the comparisons and exchanges performed during sorting.
19. Design a structure to store a student's roll number, name, and marks in three subjects. Use it to calculate and display the average marks of three students.
20. Develop a recursive C function to calculate the factorial of a given number n. Trace the recursive calls for n = 4.
21. Develop a C program to calculate the sum of the principal diagonal elements of an n × n matrix. Analyze the condition used to identify diagonal elements.
22. A program that receives a number from 1 to 7 and must display the corresponding day of the week. Select switch control structure & print the output.
23. Develop a C program to find the maximum and minimum elements in an array. Analyze the comparisons performed to obtain the two values.

24. Develop a C program to find the frequency of a search element in the given array. Analyze the comparisons performed to obtain the frequency..
25. Develop a recursive C program to reverse a given string. Trace the recursive calls for a sample string and explain the termination condition.
26. Develop a C program to search for an element in a sorted array using binary search. Analyze how the search space is reduced after each comparison.
27. Develop a C program to multiply two matrices after verifying their dimension compatibility. Analyze the condition required for matrix multiplication.
28. Develop a recursive C program to check whether a given string is a palindrome. Analyze how characters from both ends of the string are compared.
29. Develop a recursive C function to find the GCD of two integers using the Euclidean algorithm. Trace the recursive calls for a suitable pair of numbers.
30. Develop a C program using nested loops to generate the following pattern. Analyze the relationship between the outer and inner loop variables.

    5 4 3 2 1
    4 3 2 1
    3 2 1
    2 1
    1
31. Develop a recursive C function to calculate the sum of digits of a given number. Trace the execution for a sample input.
32. Develop a function to determine whether a given number is a perfect number or not.. Analyze the method used to identify and sum its proper divisors.
33. Develop a program to print all perfect nos in the given range m,n.
34. Develop a program to print all perfect nos < ,n.
35. Develop a program to print all prime nos in the given range m,n.
36. Develop a program to print all prime nos < n.
37. Write a C program using a structure to store book details such as book ID, title, author, and price. Display books whose price is greater than ₹500.
38. Read an m * n matric, find the row sum & column sum. Print the matrix along with the row sum & columnsum.
39. Write a program to sort students based on their marks using structures.
40. Write a function to calculate a^m and print the result.

****